

```
/*!
 * @brief
 * Base agnostic time system. (candidate 3)
 *
 * @assumptions
 * -> Time model uses typical hour, minute, second, etc...
 * -> Primary API impl. from derivatives
 * -> Underlying time representation is serial count of time units
 */
```

| struct TimeSystem<Time_Impl, Resolution_T, Repr_T>                                                                 |  |
|--------------------------------------------------------------------------------------------------------------------|--|
| (Friend) Time_Impl                                                                                                 |  |
| Nested Types                                                                                                       |  |
| (Public) Base = TimeSystem<Time_Impl, Resolution_T, Repr_T>                                                        |  |
| (Public) Unit_Resolution = Resolution_T                                                                            |  |
| (Public) Repr_Type = Repr_T                                                                                        |  |
| Private Attributes                                                                                                 |  |
| (Private) serialUnits: Repr_Type                                                                                   |  |
| Public Methods                                                                                                     |  |
| (Public, Static, Cexpr) toSerialSeconds(const uint8_t hr, const uint8_t min, const uint8_t sec): return -> Seconds |  |
| (Public, Static, Cexpr) hourFromSerialSecs(const Seconds serial_secs): return -> uint8_t                           |  |
| (Public, Static, Cexpr) minuteFromSerialSecs(const Seconds serial_secs): return -> uint8_t                         |  |
| (Public, Static, Cexpr) secondFromSerialSecs(const Seconds serial_secs): return -> uint8_t                         |  |
| (Public, Cexpr) operators==(const Time_Impl& time): return -> bool                                                 |  |
| (Public, Cexpr) operator<(const Time_Impl& time): return -> bool                                                   |  |
| (Public, Cexpr) operator>(const Time_Impl& time): return -> bool                                                   |  |
| (Public, Cexpr) operator<=(const Time_Impl& time): return -> bool                                                  |  |
| (Public, Cexpr) operator>=(const Time_Impl& time): return -> bool                                                  |  |
| (Public, Cexpr) operator+(const Unit_Resolution units): return -> Time_Impl                                        |  |
| (Public, Cexpr) operator++(int): return -> Date_Impl                                                               |  |
| (Public, Cexpr) operator-(const Unit_Resolution units): return -> Time_Impl                                        |  |
| (Public, Cexpr) operator-(const Time_Impl time): return -> Unit_Resolution                                         |  |
| (Public, Cexpr) operator-(int): return -> Time_Impl                                                                |  |
| (Public) operator+=(const Unit_Resolution units): return -> Time_Impl&                                             |  |
| (Public) operator++(): return -> Time_Impl&                                                                        |  |
| (Public) operator=(const Unit_Resolution units): return -> Time_Impl&                                              |  |
| (Public) operator-(): return -> Time_Impl&                                                                         |  |
| (Public, Cexpr) isZero(): return -> bool                                                                           |  |
| (Public, Cexpr) isBefore(const Time_Impl& time): return -> bool                                                    |  |
| (Public, Cexpr) isAfter(const Time_Impl& time): return -> bool                                                     |  |
| (Public, Cexpr) isBetween(const Time_Impl& start_time, const Time_Impl& end_time): return -> bool                  |  |
| (Public, Cexpr) underlying(): return -> const Repr_Type&                                                           |  |
| Private Methods                                                                                                    |  |
| (Private, Explicit, Cexpr) TimeSystem(const Repr_Type serial_units): return -> void                                |  |
| (Private) ~TimeSystem() = default -> void                                                                          |  |

```
/*!
 * @brief
 * Coordinated universal time (UTC).
 */
```

| class UTCTime : public TimeSystem<UTCTime, Seconds, Time_t>                                      |  |
|--------------------------------------------------------------------------------------------------|--|
| Public Attributes                                                                                |  |
| (Public, Static, Cexpr) standard: TimeStandard                                                   |  |
| (Public, Static, Cexpr) isCelestialTimeSystem: bool                                              |  |
| (Public, Static, Cexpr) isAtomicTimeSystem: bool                                                 |  |
| (Public, Static, Cexpr) isAtomicDerivedTime: bool                                                |  |
| Public Methods                                                                                   |  |
| (Friend) operator<=(std::ostream& os, const UTCTime& time): return -> std::ostream&              |  |
| (Public) UTCTime(const uint8_t hour, const uint8_t minute, const uint8_t second): return -> void |  |
| (Public) UTCTime(): return -> void                                                               |  |
| (Public) ~UTCTime() = default -> void                                                            |  |
| (Public, Cexpr) hour(): return -> uint8_t                                                        |  |
| (Public, Cexpr) hour12(): return -> uint8_t                                                      |  |
| (Public, Cexpr) hourPhaseStr(): return -> const char*                                            |  |
| (Public, Cexpr) hourPhaseEnumRepr(): return -> MeridiemPhase                                     |  |
| (Public, Cexpr) minute(): return -> uint8_t                                                      |  |
| (Public, Cexpr) second(): return -> uint8_t                                                      |  |
| (Public) toStr(): return -> std::string                                                          |  |
| (Public) NAME(): return -> TYPE                                                                  |  |
| Private Methods                                                                                  |  |
| (Private) NAME(): return -> TYPE                                                                 |  |

```
/*!
 * @brief
 * Base agnostic calendar system. (candidate 2)
 *
 * @assumptions
 * -> Calendar model uses a year, month, and day
 * -> Primary static API impl. from derivatives
 * -> Serial day count based calendar date impl.
 */
```

| struct CalendricalSystem<Calendar_Impl, Date_Impl, Month_Enum, DOW_Enum>                                         |  |
|------------------------------------------------------------------------------------------------------------------|--|
| (Friend) Calendar_Impl                                                                                           |  |
| Nested Types                                                                                                     |  |
| (Public) Base = CalendricalSystem<Calendar_Impl, Date_Impl, Month_Enum, DOW_Enum>                                |  |
| (Public) Date = Date_Impl                                                                                        |  |
| (Public) YearInt_t = typename Date::YearInt_t                                                                    |  |
| (Public) Month = Month_Enum                                                                                      |  |
| (Public) DayOfWeek = DOW_Enum                                                                                    |  |
| Public Methods                                                                                                   |  |
| (Public, Static, Cexpr) getMonthName(const uint8_t month): return -> const char*                                 |  |
| (Public, Static, Cexpr) getMonthName(const Month month_repr): return -> const char*                              |  |
| (Public, Static, Cexpr) getMonthName(const Date date): return -> const char*                                     |  |
| (Public, Static, Cexpr) getMonthAbbrev(const uint8_t month): return -> std::string_view                          |  |
| (Public, Static, Cexpr) getMonthAbbrev(const Month month_repr): return -> std::string_view                       |  |
| (Public, Static, Cexpr) getMonthAbbrev(const Date date): return -> std::string_view                              |  |
| (Public, Static, Cexpr) getMonthEnumRepr(const uint8_t month): return -> Month                                   |  |
| (Public, Static, Cexpr) getMonthEnumRepr(const Date date): return -> Month                                       |  |
| (Public, Static, Cexpr) getDayOfWeekName(const DayOfWeek dow_repr): return -> const char*                        |  |
| (Public, Static, Cexpr) getDayOfWeekName(const Date date): return -> const char*                                 |  |
| (Public, Static, Cexpr) getDayOfWeekAbbrev(const DayOfWeek dow_repr): return -> std::string_view                 |  |
| (Public, Static, Cexpr) getDayOfWeekAbbrev(const Date date): return -> std::string_view                          |  |
| (Public, Static, Cexpr) getDayOfWeekEnumRepr(const Date date): return -> DayOfWeek                               |  |
| (Public, Static, Cexpr) getDate(const YearInt_t year, const uint8_t month, const uint8_t day): return -> Date    |  |
| (Public, Static, Cexpr) getDate(const YearInt_t year, const Month month_repr, const uint8_t day): return -> Date |  |
| (Public, Static) getDate(const std::SystemTimePoint& time_point): return -> Date                                 |  |
| (Public, Static) getDate(const std::SystemTimePoint& time_point, const bool local): return -> Date               |  |
| (Public, Static, Cexpr) getNextDate(const Date from_date, const DayOfWeek dow_repr): return -> Date              |  |
| (Public, Static, Cexpr) getNextDate(const Date from_date, const Month month_repr): return -> Date                |  |
| (Public, Static, Cexpr) getLasDate(const Date from_date, const DayOfWeek dow_repr): return -> Date               |  |
| (Public, Static, Cexpr) getLasDate(const Date from_date, const Month month_repr): return -> Date                 |  |
| Private Methods                                                                                                  |  |
| (Private) CalendricalSystem() = delete                                                                           |  |
| (Private) ~CalendricalSystem() = delete                                                                          |  |

```
/*!
 * @brief
 * Gregorian calendar system.
 */
```

| struct GregorianCalendar : public CalendricalSystem<GregorianCalendar, GregorianCalendar, gregorian::Month, gregorian::DayOfWeek>   |  |
|-------------------------------------------------------------------------------------------------------------------------------------|--|
| Nested Types                                                                                                                        |  |
| (Public) WeekDates = std::array<Date, gregorian::DAYS_IN_WEEK>                                                                      |  |
| Public Attributes                                                                                                                   |  |
| (Public, Static, Cexpr) calendar: CalendarSystem                                                                                    |  |
| (Public, Static, Cexpr) isSolarCalendar: bool                                                                                       |  |
| (Public, Static, Cexpr) isLunarCalendar: bool                                                                                       |  |
| (Public, Static, Cexpr) isLunisolarCalendar: bool                                                                                   |  |
| (Public, Static, Cexpr) MONTH_NAMES: const std::array<const char*, gregorian::MONTHS_IN_YEAR> &                                     |  |
| (Public, Static, Cexpr) MONTH_ABBREVS: const std::array<std::string_view, gregorian::MONTHS_IN_YEAR> &                              |  |
| (Public, Static, Cexpr) DAY_OF_WEEK_NAMES: const std::array<const char*, gregorian::DAYS_IN_WEEK> &                                 |  |
| (Public, Static, Cexpr) DAY_OF_WEEK_ABBREVS: const std::array<std::string_view, gregorian::DAYS_IN_WEEK> &                          |  |
| Public Methods                                                                                                                      |  |
| (Public, Static, Cexpr) isValidYear(const YearInt_t year): return -> bool                                                           |  |
| (Public, Static, Cexpr) isLeapYear(const YearInt_t year): return -> bool                                                            |  |
| (Public, Static, Cexpr) isLeapYear(const Date date): return -> bool                                                                 |  |
| (Public, Static, Cexpr) getDaysInYear(const YearInt_t year): return -> uint16_t                                                     |  |
| (Public, Static, Cexpr) getDaysInYear(const Date date): return -> uint16_t                                                          |  |
| (Public, Static, Cexpr) isValidMonth(const uint8_t month): return -> bool                                                           |  |
| (Public, Static, Cexpr) getDaysInMonth(const YearInt_t year, const uint8_t month): return -> uint8_t                                |  |
| (Public, Static, Cexpr) getDaysInMonth(const Date date): return -> uint8_t                                                          |  |
| (Public, Static, Cexpr) isValidDay(const uint8_t day): return -> bool                                                               |  |
| (Public, Static, Cexpr) isValidDOWIndex(const uint8_t dow_index): return -> bool                                                    |  |
| (Public, Static, Cexpr) isValidDate(const YearInt_t year, const uint8_t month, const uint8_t day): return -> bool                   |  |
| (Public, Static, Cexpr) isValidDate(const Date date): return -> bool                                                                |  |
| (Public, Static, Cexpr) getDayOfWeekIndex(const YearInt_t year, const uint8_t month, const uint8_t day): return -> uint8_t          |  |
| (Public, Static, Cexpr) getDayOfWeekIndex(const Date date): return -> uint8_t                                                       |  |
| (Public, Static, Cexpr) getWeekInMonth(const YearInt_t year, const uint8_t month): return -> uint8_t                                |  |
| (Public, Static, Cexpr) getWeekInMonth(const Date date): return -> uint8_t                                                          |  |
| (Public, Static, Cexpr) getWeeksMonthSpans(const YearInt_t year, const uint8_t month): return -> uint8_t                            |  |
| (Public, Static, Cexpr) getWeeksMonthSpans(const Date date): return -> uint8_t                                                      |  |
| (Public, Static, Cexpr) toDaysSinceEpoch(const YearInt_t year, const uint8_t month, const uint8_t day): return -> Days              |  |
| (Public, Static, Cexpr) toDaysSinceEpoch(const Date date): return -> Days                                                           |  |
| (Public, Static, Cexpr) fromDaysSinceEpoch(const Days serial_days): return -> Date                                                  |  |
| (Public, Static, Cexpr) toUnixTimestamp(const YearInt_t year, const uint8_t month, const uint8_t day): return -> std::UnixTimestamp |  |
| (Public, Static, Cexpr) toUnixTimestamp(const Date date): return -> std::UnixTimestamp                                              |  |
| (Public, Static, Cexpr) fromUnixTimestamp(const std::UnixTimestamp serial_secs): return -> Date                                     |  |
| (Public, Static, Cexpr) getNextWeekday(const Date from_date): return -> Date                                                        |  |
| (Public, Static, Cexpr) getLasWeekday(const Date from_date): return -> Date                                                         |  |
| (Public, Static, Cexpr) getNextWeekend(const Date from_date): return -> Date                                                        |  |
| (Public, Static, Cexpr) getLasWeekend(const Date from_date): return -> Date                                                         |  |
| (Public, Static, Cexpr) getDatesInWeek(const YearInt_t year, const uint8_t week_index): return -> WeekDates                         |  |
| (Public, Static, Cexpr) getDatesInWeek(const Date date): return -> WeekDates                                                        |  |
| Private Methods                                                                                                                     |  |
| (Private) GregorianCalendar() = delete                                                                                              |  |
| (Private) ~GregorianCalendar() = delete                                                                                             |  |

```
/*!
 * @brief
 * Base calendar agnostic date. (candidate 2)
 *
 * @assumptions
 * -> Calendar model uses a year, month, and day
 * -> Primary API impl. from derivatives
 * -> Underlying date representation is serial day count
 */
```

| struct CalendarDate<Date_Impl, Calendar_Policy, Repr_T, Year_T>                                   |  |
|---------------------------------------------------------------------------------------------------|--|
| (Friend) Date_Impl                                                                                |  |
| Nested Types                                                                                      |  |
| (Public) Base = CalendarDate<Date_Impl, Calendar_Policy, Repr_T, Year_T>                          |  |
| (Public) Repr_Type = Repr_T                                                                       |  |
| (Public) YearInt_t = Year_T                                                                       |  |
| (Public) CalendarPolicy = Calendar_Policy                                                         |  |
| Private Attributes                                                                                |  |
| (Private) serialDays: Repr_Type                                                                   |  |
| Public Methods                                                                                    |  |
| (Friend) operator<=(std::ostream& os, const Base& date): return -> std::ostream&                  |  |
| (Public, Cexpr) operators==(const Date_Impl& date): return -> bool                                |  |
| (Public, Cexpr) operator<(const Date_Impl& date): return -> bool                                  |  |
| (Public, Cexpr) operator>(const Date_Impl& date): return -> bool                                  |  |
| (Public, Cexpr) operator<=(const Date_Impl& date): return -> bool                                 |  |
| (Public, Cexpr) operator>=(const Date_Impl& date): return -> bool                                 |  |
| (Public, Cexpr) operator+(const Days days): return -> Date_Impl                                   |  |
| (Public, Cexpr) operator++(int): return -> Date_Impl                                              |  |
| (Public, Cexpr) operator-(const Days days): return -> Date_Impl                                   |  |
| (Public, Cexpr) operator-(const Date_Impl date): return -> Days                                   |  |
| (Public, Cexpr) operator-(int): return -> Date_Impl                                               |  |
| (Public) operator+=(const Days days): return -> Date_Impl&                                        |  |
| (Public) operator++(): return -> Date_Impl&                                                       |  |
| (Public) operator=(const Days days): return -> Date_Impl&                                         |  |
| (Public) operator-(): return -> Date_Impl&                                                        |  |
| (Public, Cexpr) isZero(): return -> bool                                                          |  |
| (Public, Cexpr) isBefore(const Date_Impl& date): return -> bool                                   |  |
| (Public, Cexpr) isAfter(const Date_Impl& date): return -> bool                                    |  |
| (Public, Cexpr) isBetween(const Date_Impl& start_date, const Date_Impl& end_date): return -> bool |  |
| (Public, Cexpr) daysUntil(const Date_Impl& date): return -> Days                                  |  |
| (Public, Cexpr) underlying(): return -> const Repr_Type&                                          |  |
| Private Methods                                                                                   |  |
| (Private, Explicit, Cexpr) CalendarDate(const Repr_Type serialDayCount): return -> void           |  |
| (Private) ~CalendarDate() = default -> void                                                       |  |
| (Private, Cexpr) thisImpl(): return -> const Date_Impl&                                           |  |
| (Private) thisImpl(): return -> Date_Impl&                                                        |  |

```
/*!
 * @brief
 * Gregorian calendar date.
 */
```

| struct GregorianCalendar : public CalendarDate<GregorianCalendar, GregorianCalendarPolicy, int32_t, gregorian::Year_Type> |  |
|---------------------------------------------------------------------------------------------------------------------------|--|
| Public Attributes                                                                                                         |  |
| (Public, Static, Cexpr) NAME: TYPE                                                                                        |  |
| Public Methods                                                                                                            |  |
| (Public, Cexpr) GregorianCalendar(const YearInt_t year, const uint8_t month, const uint8_t day): return -> void           |  |
| (Public, Cexpr) GregorianCalendar(): return -> void                                                                       |  |
| (Public) ~GregorianCalendar() = default -> void                                                                           |  |
| (Public, Cexpr) year(): return -> YearInt_t                                                                               |  |
| (Public, Cexpr) month(): return -> uint8_t                                                                                |  |
| (Public, Cexpr) day(): return -> uint8_t                                                                                  |  |
| (Public) toStr(): return -> std::string                                                                                   |  |
| (Public) NAME(): return -> TYPE                                                                                           |  |
| Private Methods                                                                                                           |  |
| (Public) NAME(): return -> TYPE                                                                                           |  |